

Lotto 사이트 과제 보고서

1. 개요

본 프로젝트는 Django와 Docker를 사용하여 6/45 Lotto 웹 사이트를 구현한 과제입니다. 일반 사용자는 회원권을 구매할 수 있고, 추첨 이후 본인의 당첨 여부를 확인할 수 있습니다. 관리자는 Django admin을 통해 판매 추첨을 실행하며 당첨 결과를 관리합니다.

- GitHub 링크: <https://github.com/PilSun100/Lotto>
- 구현 기술: Django, PostgreSQL, Docker Compose, HTML/CSS
- 컨테이너 구성: `web` Django 애플리케이션, `db` PostgreSQL 데이터베이스

2. 시스템 설계

서비스는 Django 프로젝트 `config`와 두 개의 앱으로 구성했습니다.

- `accounts`: Django 기본 인증을 활용한 회원가입, 로그인, 로그아웃 기능
- `lottery`: 회차, 복권, 추첨 결과 모델과 구매/당첨 확인 화면

주요 데이터 모델은 다음과 같습니다.

- `Round`: 회차 번호, 판매 시작/종료 시각, 추첨 완료 여부 저장
- `Ticket`: 구매 사용자, 회차, 선택 번호, 자동/수동 구매 방식 저장
- `DrawResult`: 회차별 당첨 번호 6개, 보너스 번호, 추첨 시각 저장

데이터베이스는 Docker 환경에서 PostgreSQL을 사용하고, 로컬 개발 및 테스트 환경에서는 별도 설정 없이 SQLite로 구성했습니다.

3. 구현 과정

일반 사용자 기능은 Django 템플릿 기반 웹 화면으로 구현했습니다. 사용자는 `/accounts/signup/`에서 가입 후 `/buy/`에서 복권을 구매합니다. 수동 구매는 1부터 45 사이의 중복 없는 숫자 6개를 입력해야 하며, 자동 구매 `sample`을 사용해 중복 없는 번호 6개를 생성합니다. 추첨 이후에는 `/tickets/check/` 화면에서 최근 당첨 결과를 확인할 수 있습니다.

관리자 기능은 Django admin 중심으로 구현했습니다. 관리자는 `Round`를 생성하여 판매 기간을 열 수 있고, `DrawResult`를 생성하여 `Round`에서 선택한 회차 추첨 실행을 실행해 당첨 번호와 보너스 번호를 랜덤 생성합니다. `DrawResult`는 추첨 결과만 보기 전용으로 제한하여, 모든 당첨 번호가 회차별 추첨 기능을 통해서만 생성되도록 했습니다. 추첨이 완료되면 `내 복권` 화면에서 당첨 등수가 표시됩니다.

당첨 등수 계산 기준은 다음과 같습니다.

- 1등: 당첨 번호 6개 일치
- 2등: 당첨 번호 5개와 보너스 번호 일치
- 3등: 당첨 번호 5개 일치
- 4등: 당첨 번호 4개 일치
- 5등: 당첨 번호 3개 일치
- 미당첨: 그 외

4. Docker 구성

`docker-compose.yml`은 두 개의 컨테이너로 구성했습니다.

- `db`: `postgres:16-alpine` 이미지 사용
- `web`: 프로젝트 `Dockerfile`로 빌드하며 Django migration 후 Gunicorn 실행

실행 명령은 다음과 같습니다.

```
```bash
cp .env.example .env
docker compose up --build
```
```

관리자 계정 생성은 다음 명령으로 수행합니다.

```
```bash
docker compose exec web python manage.py createsuperuser
```
```

5. 테스트 결과

다음 테스트를 작성했습니다.

- 자동 번호 생성이 1~45 범위, 6개, 중복 없음 조건을 만족하는지 검증
- 수동 번호 입력의 개수 부족, 중복, 범위 초과 실패 케이스 검증
- 1등부터 5등 및 미당첨 당첨 등수 계산 검증
- 로그인 사용자의 수동/자동 복권 구매 검증
- 사용자가 본인의 복권만 조회할 수 있는지 검증
- 추첨 전후 당첨 결과 표시 변화 및 당첨 확인 화면 검증
- Django admin action으로 추첨 결과가 생성되는지 검증
- 추첨 결과 admin에서 직접 추가가 차단되는지 검증

실행 명령:

```
```bash
python manage.py test
```
```

Docker 환경 테스트 명령:

```
```bash
docker compose exec web python manage.py test
```
```

6. AI 도구 사용 내역

본 과제 수행 중 ChatGPT/Codex를 사용했습니다. 사용 내역은 다음과 같습니다.

- 과제 요구사항 분석 및 구현 계획 수립 보조
- Django 모델, 폼, 뷰, admin action, 테스트 코드 작성 보조
- Docker Compose 구성 및 실행 문서 작성 보조
- 보고서 초안 작성 및 제출 항목 누락 여부 점검 보조

최종 코드는 요구사항에 맞게 검토하고 테스트를 통해 동작을 확인했습니다.